Challenges in Delivering Software in the Cloud as Microservices

Christian Esposito
University of Naples "Federico II"

Aniello Castiglione
University of Salerno

Kim-Kwang Raymond Choo
University of Texas at San Antonio

MICROSERVICES, ONE OF THE LATEST ARCHITECTURAL TRENDS IN SOFTWARE ENGINEERING,[1] CAN BE BROADLY DEFINED AS THE DESIGN OF SERVICE-ORIENTED SOFTWARE USING A SET OF SMALL SERVICES. These services are small applications that can be deployed independently, with a precise and hardened interface, and easily integrated. Microservices are supported by middleware for communication and a platform for flexible and low-cost deployment.

The small applications have a high degree of internal cohesion around a single task and can cope with a simple responsibility. Here, we consider "responsibility" as doing (or being responsible for) one activity only. The activity can include serving or representing a particular resource. Practical examples include logging incoming messages within a database or a file, or managing a message queue by taking, processing, and discarding queued messages. Such a concept started within the industrial practice of splitting large monolithic applications into small cooperating pieces to improve their maintainability, scalability, and testability. Microservice architectures are receiving increasing focus, as evidenced by search statistics on Google Trends.[2] Academia's interest in microservices is also evidenced by the publication of the first mature academic book in 2015.[3]

The increasing popularity of microservices, despite the effort required to implement them, is probably due to the benefits associated with their agility, resilience, scalability, and maintainability.[4] Specifically, we could enforce separation of concerns by applying the single responsibility principle.

Microservices in the Cloud

Recent advancements in container technologies and the capability to overcome limitations in virtualization have, perhaps, encouraged the use of containers in the cloud for software development and deployment. This might also have paved the way for the adoption of a microservice architectural paradigm in cloud-hosted software by lowering infrastructure and maintenance costs.[2,5,6] For example, microservices support the realization of small (sized) applications that are fine-grained and loosely coupled and communicate through REST protocols. These applications are implemented using APIs provided by the infrastructure-as-a-service (IaaS) layer for provisioning data computing, storage, and delivery capabilities.

 2325-6095/16/$33.00 © 2016 IEEE

The growing adoption of microservices in the cloud is motivated by the ease of deploying and updating the software, as well as the provisioned loose coupling provided by dynamic service discovery and binding. Moreover, structuring the software to be deployed in the cloud as a collection of microservices allows cloud service providers to offer higher scalability guarantees through more efficient consumption of cloud resources, and to dynamically and quickly restructure software to accommodate growing consumer demand. Structuring software in smaller computation units enables optimized allocation of the application components within proper containers in the virtual machines (VMs) running on top of the host machine provided by the cloud infrastructure, as the example in Figure 1 illustrates. This lets us minimize waste of resources and maximize packing of the components within a single VM. This is possible because, despite having to realize the same application tasks, microservices are typically thinner than conventional software components because they use lightweight software technologies and platforms. This model also allows a simpler and faster migration of software component instances from one VM to another to satisfy cloud applications' changing resource demands.

Partitioning monolithic applications into small pieces of computation also allows for the segmentation of application data, and thus avoids having a single monolithic data storage. Such a solution is required by the decentralized data governance that a microservice architecture imposes and is known in the literature as the sharding pattern.[7] It consists of dividing a data store into a set of horizontal partitions, or shards, to improve scalability and performance when storing and accessing large volumes of data, as in the case of cloud-hosted applications. The different shards are kept consistent with respect to a centralized data storage, or through a distributed protocol.

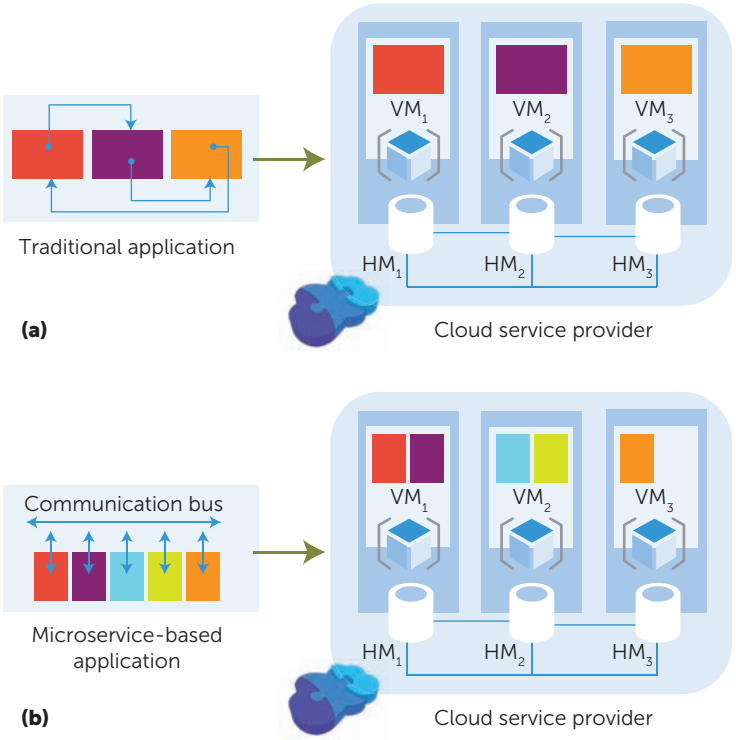

FIGURE 1. Software deployment in a cloud platform using (a) conventional and (b) microservice-based software.

Security Threats and Legal Ramifications

The key idea underlying the microservice architecture is to distribute application complexity among narrowly focused and independently deployable units of computation. Such small components are loosely coupled by means of an integration middleware that dynamically and seamlessly resolves the implicit dependencies among them.

Figure 2 depicts the differences between a conventional monolithic architecture and a microservice architecture. In the first case (Figure 2a), users interact with a front-end application, which redirects user requests to multiple instances of the software hosted within a container and implements all application responsibility using the data stored in a database. In the second case (Figure 2b), the software is split into multiple components, each implementing only its given responsibility, hosted in multiple containers, and/or replicated at the user's convenience. The dependencies among the microservices and data shards are complex and having centralized data storage can help maintain consistency. Using existing middleware solutions, such as those offering event notification,[8] allows such architectures to cope with the complexity of connecting multiple entities, even with the dynamic discovery of components.

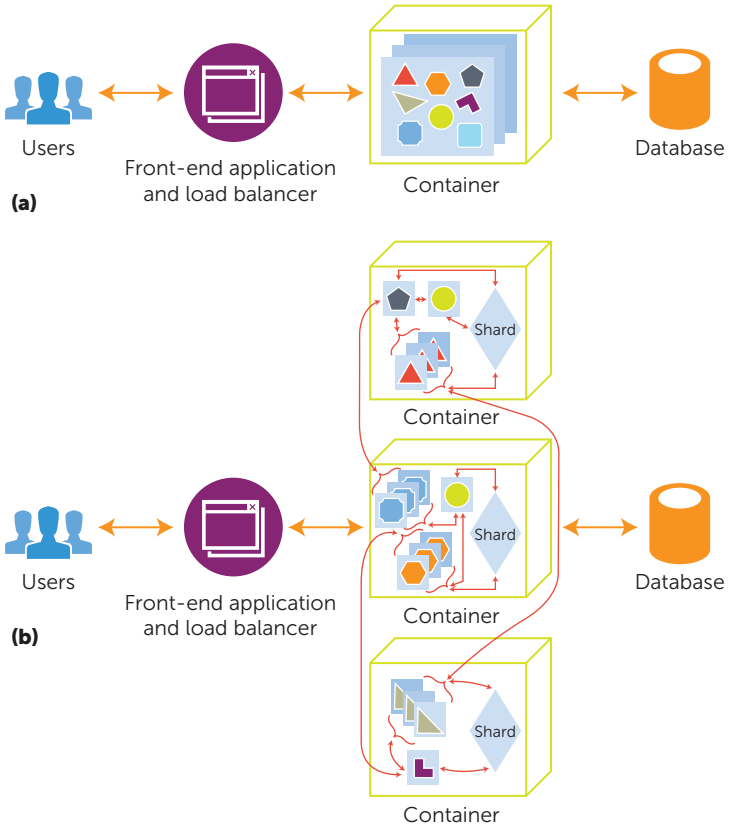

FIGURE 2. Architectural complexity of (a) monolithic and (b) microservice-based software.

However, such complexity can result in security vulnerabilities affecting one or more vectors in the architecture,[9] including single microservices,[10] data shards,[11] and the cooperation and orchestration among microservices.[12] In the first vulnerability vector, having multiple components within an architecture can enlarge the attack surface by presenting more points of vulnerability that can be exploited to compromise the application. In addition, the complexity of gluing together a multitude of microservices complicates the challenge of debugging, monitoring, and auditing the application, especially within a cloud that's not under the application owner's control. Moreover, in microservice architectures, reuse is strongly enforced by allowing the use of off-the-shelf (OTS) components. Deployed OTS components represent a security threat and they should be properly validated, authenticated, and monitored.[13]

Trustworthiness is also an issue when dealing with microservices. For example, an adversary could compromise or gain control of a component, which isn't uncommon within the public cloud context. However, in a typical microservice architecture, the other components assume a trusted component base; thus, there's a real risk that attacks can be easily propagated due to the microservices' dependencies and (blind) trust of peer components. In data sharding, security threats are related to data provenance, trustworthiness, and protection. Specifically, when data is horizontally segmented into multiple shards, we need to determine if the data source is verifiable (or trustworthy) and how to protect against forging attempts (for example, data modification and tampering) during sharding.

Furthermore, data can be provided by multiple data sources, and each should have a reputation degree associated with it. This would give data consumers a way to determine the trustworthiness degree of the data prior to using it in their computations. Data must also be kept consistent with respect to eventual replicas (or a centralized data store), since such a process is vulnerable to exposure, denial of service, and alteration attacks. Even in the best-case scenario, where we assume a single microservice has been demonstrated to be secure and the data sharding mechanism protected, the architecture presents an additional point of vulnerability: the composition and coordination middleware can also be targeted and exploited. In the absence of a centralized orchestrator, for example, microservices coordinate among themselves through message-based communications. These communications, if not protected, won't let us achieve secure guarantees during data exchange and/or receive falsified or modified coordination which is required when undertaking complex and elaborate functionalities.[14]

Should a breach or cybersecurity incident occur as a result of a successful exploitation in any of the three vectors (single microservices, data shards, and the cooperation and orchestration among microservices), who is responsible for users' pure economic loss (such as direct and indirect financial loss) under the current legal framework—the microservice provider or the cloud service provider? If an attacker exploits the "trust" in peer components within the architecture, should the compromised

microservice provider be held responsible? What happens if the compromised microservice provider is found to have failed to provide reasonable industry-standard security?

Because microservices are relatively new, we don't know if there are gaps in existing legal frameworks. For example, because there are no known cases on which to comment, we can't effectively assess the adequacy of existing legislation, and challenges will arise as untested provisions are applied in civil litigation or prosecution. Thus, we need regular dialogues between service providers (such as microservices and cloud services), legal practitioners, and policymakers to improve understanding of the nature and extent of the risks and associated legal implications. This would, in turn, guide further responses at the operational and strategic levels for service providers and provide the evidence base to inform policymakers in governments to design national regulatory strategies to address the associated risks more effectively (for example, without resulting in a floodgate of claims against service providers).

Mitigation

Conventional solutions such as secure message-passing middleware, physical security for cloud infrastructure, and privacy-preserving cloud data storage schemes might only be effective for one or more components, but they're unlikely to address the security (and certainly not the legal) challenges due to microservices' unique characteristics.

Potential research to address these technical limitations or security vulnerabilities can focus on several areas. One area is microservice validation, which could be conducted both in isolation and in composition with other microservices, allowing us to identify and mitigate potential vulnerabilities in the software, implementation, or interaction between microservices.

In addition, we need a dynamic, and preferably lightweight, security monitoring and management system, responsible for monitoring and enforcing correct behavior of microservices and other related activities (such as cooperation). Once the behavior or activities deviate from the norm, the system should undertake corrective actions and/or countermeasures to bring the application to the correct state by autonomously resolving possible compromises and violations.

We also need proper cryptographic schemes for data protection at rest and in motion, allowing access and integrity control to the exchanged messages and stored shards while allowing privacy-preserving processing tasks.

WE NEED TO STRIKE A BALANCE BETWEEN SECURITY AND PERFORMANCE. For example, security mitigation solutions should also provide important nonfunctional requirements, such as minimal overhead and tamper resistance, so they minimally impact the system's performance and users' quality of experience. Ideally, the security solutions should continue to function after an attacker has successfully taken control of the cloud infrastructure or VMs. Since equipping microservices with a proper security mechanism might not always be the best option,[10] offering security as a service for microservice-based cloud applications can be an attractive solution.

References

1. M. Fowler, "Microservices Resource Guide," 2016; http://martinfowler.com/microservices.
2. A. Balalaie, A. Heydarnoori, and P. Jamshidi, "Microservices Architecture Enables DevOps," *IEEE Software*, vol. 33, no. 3, 2016, pp. 42–52.
3. S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O'Reilly Media, 2015.
4. T. Killalea, "The Hidden Dividends of Microservices," *Comm. ACM*, vol. 59, no. 8, 2016, pp. 42–45.
5. F. Oliveira et al., "Delivering Software with Agility and Quality in a Cloud Environment," *IBM J. Research and Development*, vol. 60, nos. 2–3, 2016, pp. 10:1-10:11.
6. M. Villamizar et al., "Infrastructure Cost Comparison of Running Web Applications in the Cloud Using AWS Lambda and Monolithic and Microservice Architectures," *Proc. 16th IEEE/ACM Int'l Symp. Cluster, Cloud and Grid Computing* (CCGrid), 2016, pp. 179–182.
7. C.H. Costa et al., "Sharding by Hash Partitioning: A Database Scalability Pattern to Achieve Evenly Sharded Database Clusters," *Proc. 17th*

Int'l Conf. Enterprise Information Systems (ICEIS), 2015.
8. C. Esposito, D. Cotroneo, and S. Russo, "On Reliability in Publish/Subscribe Services," *Computer Networks*, vol. 57, no. 5, 2013, pp. 1318–1343.
9. N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," 2016; https://arxiv.org/abs/1606.04036.
10. Y. Sun, S. Nanda, and T. Jaeger, "Security-as-a-Service for Microservices-Based Cloud Applications," *Proc. 7th IEEE Int'l Conf. Cloud Computing Technology and Science* (CloudCom), 2015, pp. 50–57.
11. F. Callegati et al., "Data Security Issues in MaaS-Enabling Platforms," *Proc. Int'l Forum Research and Technologies for Society and Industry*, 2016; https://hal.inria.fr/hal-01336700.
12. S. Kim et al., "High-Assurance Synthesis of Security Services from Basic Microservices," *Proc. 14th Int'l Symp. Software Reliability Eng.* (ISSRE), 2003, pp. 154–165.
13. D.I. Savchenko, G.I. Radchenko, and O. Taipale, "Microservices Validation: Mjolnirr Platform Case Study," *Proc. 38th Int'l Convention Information and Comm. Technology, Electronics and Microelectronics* (MIPRO), 2015, pp. 235–240.
14. C. Esposito and M. Ciampi, "On Security in Publish/Subscribe Services: A Survey," *IEEE Comm. Surveys and Tutorials*, vol. 17, no. 2, 2015, pp. 966–997.

CHRISTIAN ESPOSITO *is adjunct professor at the University of Naples "Federico II," Italy, and research fellow and adjunct professor at the University of Salerno, Italy. His research interests include information security and reliability, middleware, and distributed systems. Esposito has a PhD in computer engineering from the University of Naples "Federico II," Italy. Contact him at christian.esposito@dia.unisa.it.*

ANIELLO CASTIGLIONE *is adjunct professor at the University of Salerno, Italy, and at the University of Naples "Federico II," Italy. His research interests include security, communication networks, information forensics and security, and applied cryptography. Castiglione has a PhD in computer science from the University of Salerno, Italy. He's member of several associations, including IEEE and ACM. Contact him at castiglione@ieee.org.*

KIM-KWANG RAYMOND CHOO *holds the Cloud Technology Endowed Professorship at the University of Texas at San Antonio. His research interests include cyber and information security and digital forensics. Choo has a PhD in information security from Queensland University of Technology, Australia. He's a fellow of the Australian Computer Society and a senior member of IEEE. Contact him at raymond.choo@fulbrightmail.org.*

IEEE computer society

IEEE

2017 B. Ramakrishna Rau Award

Call for Nominations

Honoring contributions to the computer microarchitecture field

New Deadline: 1 May 2017

Established in memory of Dr. B. (Bob) Ramakrishna Rau, the award recognizes his distinguished career in promoting and expanding the use of innovative computer microarchitecture techniques, including his innovation in complier technology, his leadership in academic and industrial computer architecture, and his extremely high personal and ethical standards.

WHO IS ELIGIBLE? The candidate will have made an outstanding innovative contribution or contributions to microarchitecture, use of novel microarchitectural techniques or compiler/architecture interfacing. It is hoped, but not required, that the winner will have also contributed to the computer microarchitecture community through teaching, mentoring, or community service.

AWARD: Certificate and a $2,000 honorarium.

PRESENTATION: Annually presented at the ACM/IEEE International Symposium on Microarchitecture

NOMINATION SUBMISSION: This award requires 3 endorsements. Nominations are being accepted electronically: www.computer.org/web/awards/rau

CONTACT US: Send any award-related questions to awards@computer.org

www.computer.org/awards

myCS

Read your subscriptions through the myCS publications portal at **http://mycs.computer.org.**